

EECE.4520 Microprocessor System II and Embedded System Design

Theodore Skafidas

02022321

Team T

Date of Completion: November 2, 2025

Demonstration Method: in person

Lab 2

Game Control Joystick, Gyro and Accelerometer

I. DESIGN

This section outlines the hardware and software design used to create the game controller for the Python “snake” game. The system is split into two main components: the host computer running the Python game logic and the Arduino Mega 2560 microcontroller responsible for processing all sensor and actuator information. These two components communicate via a bi-directional serial port.

A. Hardware Design

The hardware design used an ATmega2560 microcontroller to communicate with the physical components. The microcontroller interfaces with a 2-axis analog joystick, an MPU-6500 6-axis gyro/accelerometer, and a passive buzzer. The joystick X-axis (VRx) and Y-axis (VRy) analog outputs are connected to the microcontroller’s analog input pins A0 and A1, respectively. This allows the controller to read the stick’s position as an integer value from 0-1023. The MPU-6500 sensor communicates with the microcontroller using the I2C protocol. The sensor’s serial data (SDA) and serial clock (SCL) pins are connected to the dedicated I2C pins on the ATmega2560.

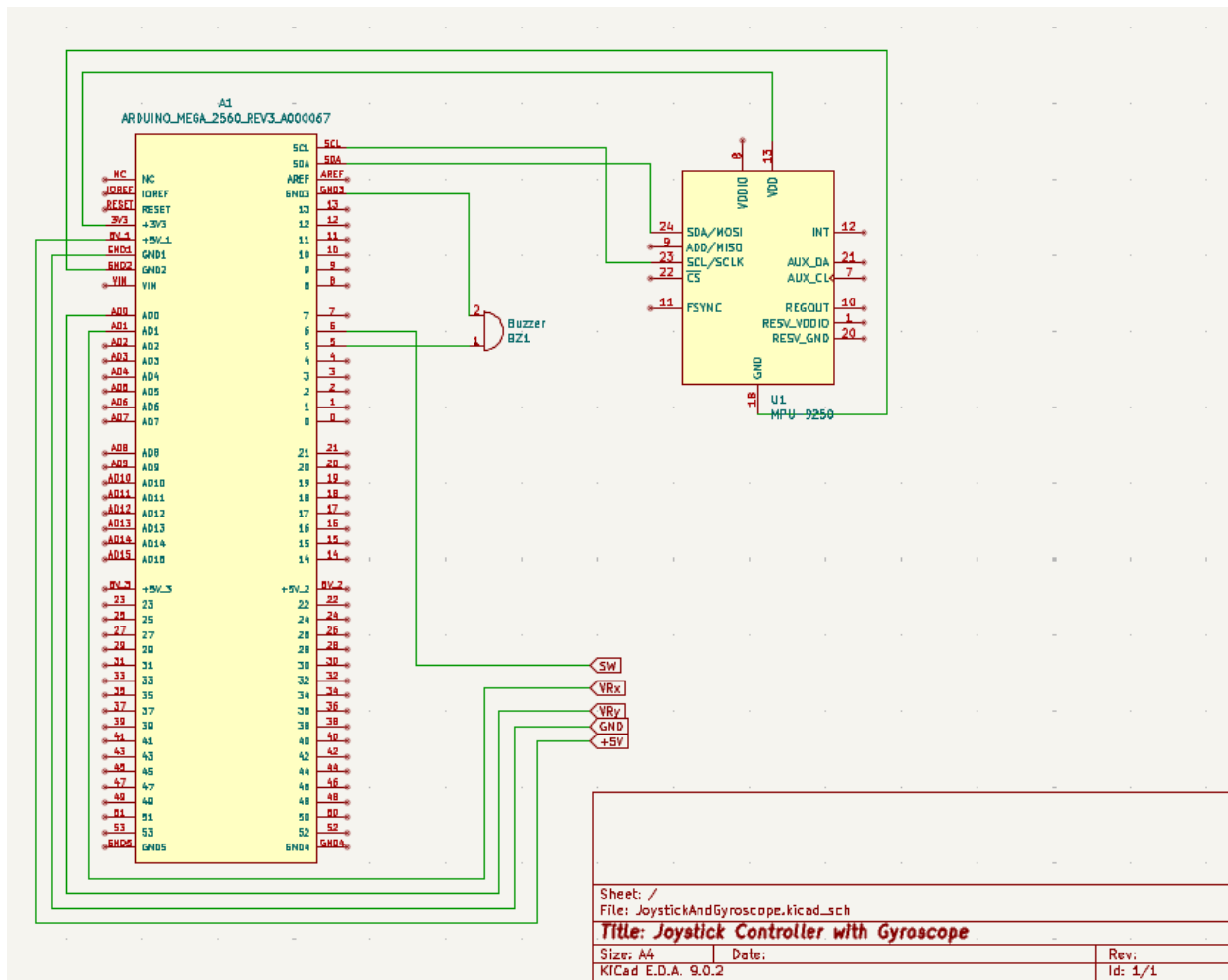


Figure 1 – Joystick Controller Schematic

The global connectors are meant to correspond to the pins connected to the physical joystick. The names for these pins are directly translated into the full schematic for a more complete picture.

B. Software Design

The following link contains the complete code used in this lab:

<https://github.com/Tf3tus/Lab2-GameController.git>

1. Arduino Controller (Gyro-Serial-RW.ino)

The Arduino software was designed as a continuous polling loop that serves as the hardware abstraction layer for the game. Instead of a complex finite-state machine, this design is event-driven, checking for sensor input and serial commands on every loop.

The main loop() continuously performs two independent tasks:

- 1. Check for Host Commands:** It checks if serial data is available (`Serial.available() > 0`). If the incoming byte is 'A', it calls the `beepBuzzer()` function.
- 2. Check Sensors:** To prevent spamming the serial port, a non-blocking timer (`millis()`) is used to ensure sensor data is only read and sent every 100ms.

Sensor Polling (Gyro): To provide a stable tilt control, the MPU9250_WE library was used. On startup, the sensor is calibrated using `myMPU9250.autoOffsets()`. In the loop, `getPitch()` and `getRoll()` are called to get the sensor's absolute angle, a result of the library's built-in sensor function. This angle is compared against a "deadzone" (`angleThreshold`) to determine if a 'w', 'a', 's', or 'd' command should be sent. This absolute angle, rather than raw gyro velocity, was a critical to solving the "return-to-center" bug.

Actuator Control: The `beepBuzzer()` function uses the `tone()` and `noTone()` commands to generate the specific frequency required to drive the passive buzzer.

2. Host Game (snake.py)

The host software is a modified Python Turtle game. The `pyserial` library was integrated to establish communication with the microcontroller.

Receiving Controls: With the main `while True:` game loop, the script performs a non-blocking check for incoming serial data (`ser.in_waiting > 0`). If a line is received, it's decoded (`.decode('utf-8').strip()`), and the resulting string ('w', 'a', etc.) is used to call the corresponding `go_up()`, `go_down()`, etc., functions to control the snake.

Sending Events: When the game detects a collision between the snake's head and the food (`head.distance(food) < 20`), it immediately sends the byte `b'A'` over the serial port (`ser.write(b'A')`). This signals the Arduino to activate the buzzer.

The following flowcharts illustrate the program logic for both the Arduino controller and the Python game.

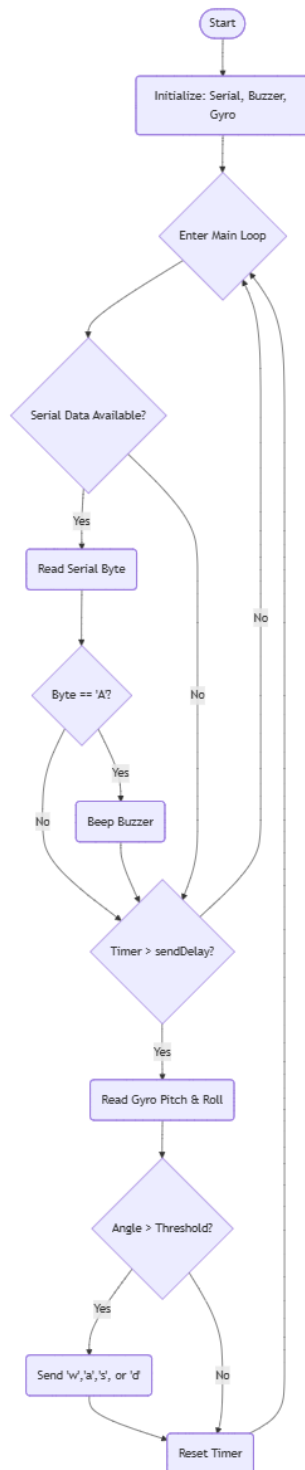


Figure 2 – Arduino Controller Flowchart

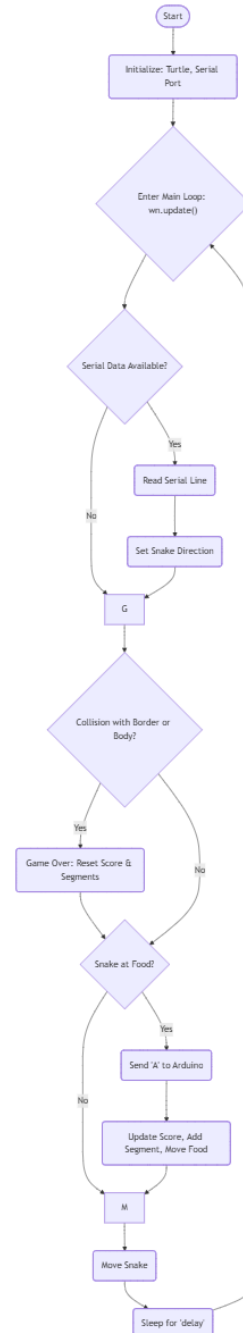


Figure 3 – Python Game Flowchart

C. Results

The game controller was successfully implemented and met all functional requirements outlined in the lab. The system correctly operates according to the flowcharts. The figure below shows the completed hardware setup.

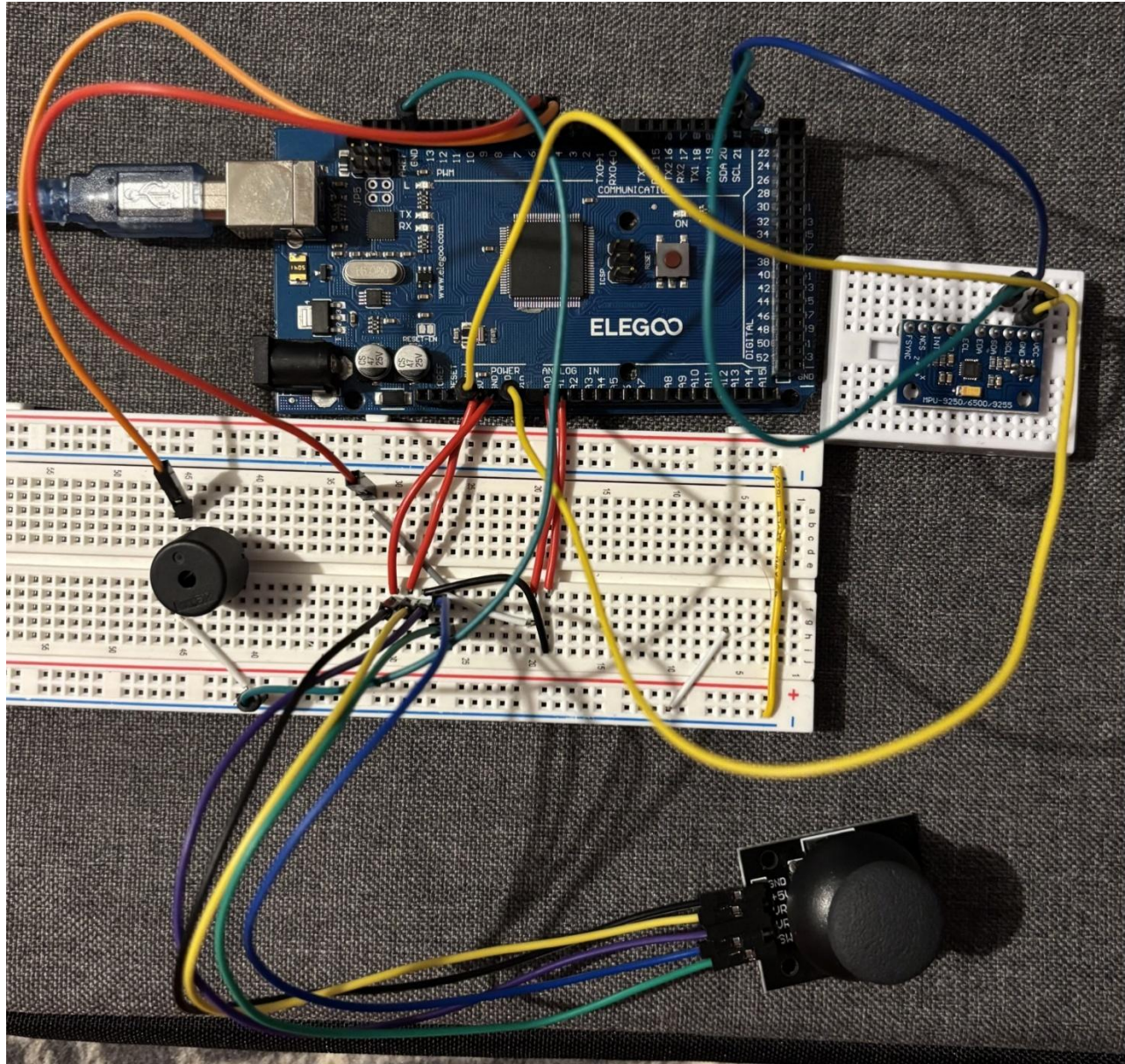


Figure 4 – Physical Game Controller

The following results correspond to the tasks performed by the controller and host game.

1. Serial Communication

The pyserial library and Arduino Serial library established a stable, bi-directional communication link at 115200 baud.

2. *Joystick Control*

The joystick controller (Serial-RW.ino) successfully controlled the snake. The analogRead() values were correctly mapped, with a deadzone of 300-700 preventing drift when the stick was centered.

3. *Gyroscope Control*

The gyro controller (Gyro-Serial-RW.ino) also successfully controlled the snake. By using the MPU9250_WE library's getPitch() and getRoll() functions, the snake's movement was based on the sensor's absolute tilt angle, providing an intuitive control scheme.

4. *Buzzer Control*

When the snake ate the apple, the snake.py script correctly sent the b'A' byte. The Arduino received this byte and triggered the beepBuzzer() function, providing immediate audio feedback.

II. PROBLEMS

During the development and testing of the game controller, several challenges were encountered. This section documents the key problems and the debugging methods used to resolve them.

A. *Properly Read the Gyroscope Input*

Problem: Movements made to return the gyroscope to its zero position would be read as input for the program. For example, when controlling the snake with the gyroscope, tilting the sensor forward correctly sent 'w', but tilting it *back to the starting position* (center) incorrectly sent 's'. The snake should only move backward when tilted *past* the center.

Solution: The root cause was using the raw gyroscope velocity data (readGyro()). This function measures the rate of rotation, so tilting back to center is a negative velocity, which the code interpreted as a "down" command. The solution was to use a combination of the gyroscope and accelerometer data. By using the MPU9250_WE library and its getPitch() and getRoll() functions, we were able to get the sensor's absolute tilt angle. By comparing this angle to a threshold (angleThreshold = 20.0), the code now only sends a command when the sensor is physically tilted beyond 20 degrees, solving the bug.

B. *Failure to Produce Sound in Buzzer*

Problem: The buzzer pin would receive a HIGH signal, but no sound was produced.

Solution: Research determined the difference between active and passive buzzers. The component used was a passive buzzer, which requires an oscillating signal (a square wave) to generate sound, whereas the initial code produces only a steady DC voltage via digitalWrite(). The problem was solved by replacing the initial calls with the Arduino tone() and noTone() functions, which generated the necessary frequency to drive the passive buzzer.

C. *Installing Third-Party Hardware Library*

Problem: The provided sensor was an MPU-6500, which required a library not included with the Arduino IDE.

Solution: A correct library was found on GitHub, and the library was downloaded as a .ZIP file from the repository's main page. Then the library was installed to the IDE using the Include Library feature.

III. TECHNICAL DETAILS

As the sole contributor to this project, I was responsible for all aspects of the design, implementation, and testing of the game controller.

Hardware Design and Assembly:

- Designed the complete circuit schematic, describing the connections between the ATmega2560, joystick, MPU96500, and buzzer.
- Physically assembled and wired the entire circuit on a breadboard.

Software Design (Arduino):

- Wrote the complete Serial-RW.ino sketch for the joystick controller, including the logic to read analog pins and map their values to direction commands with a software deadzone.
- Wrote the complete Gyro-Serial-RW.ino sketch to handle the gyroscope controller.
- Researched and integrated the MPU9250_WE third-party library to interface with the MPU6500 sensor.
- Implemented the sensor calibration (autoOffsets()) and sensor fusion logic (getPitch() and getRoll()) to provide stable, absolute-angle tilt control.
- Wrote the non-blocking timer logic using millis() to prevent serial spam and ensure the main loop remained responsive.

Software Design (Python):

- Modified the snake.py script to fully integrate the pyserial library for bi-directional serial communication.
- Implemented the serial-reading logic in the main game loop to parse incoming bytes ('w', 'a', 's', 'd') and mapped them to the corresponding snake direction functions.
- Implemented the serial-writing logic to send a b'A' byte (for "apple eaten") to the Arduino when a food collision was detected.

Debugging and Problem Solving:

- Debugged the entire system from end-to-end. This included using the serial monitor to trace program flaws.
- Found and resolved multiple integration issues between the hardware and software, namely the MPU6500 and the buzzer.

IV. CONCLUSION

This lab provided significant hands-on experience in integrating a microcontroller with a PC application. One primary lesson was the practical application of serial communication for command and control. The key takeaway was understanding the host-client relationship and the protocol required for the two systems to interact. A critical hardware rule was also discovered: only one application can control a COM port at a time. Similar to the previous lab, this lab reinforced the importance of non-blocking code. Using `millis()` was essential to read the incoming serial data while waiting to send sensor data. The most significant lesson was understanding the difference between raw sensor data (velocity) and the processed data (angle). The raw gyro data is noisy and measures the rate of change, while combining the gyro and accelerometer data provides a stable, absolute angle that is far more useful for tilt-based controls. Finally, the project reinforced the routine method of debugging, such as serial tracing, to pinpoint and solve software integration problems.

V. REFERENCES

- [1] *Lab 2: Game Control with Joy Stick, Gyro and Accelerometer*, Luo.Y, 2025
- [2] *eece-4520-5520-labs*, Luo.Y, GitHub, 2025. <https://github.com/ACANETS/eece-4520-5520-labs>
- [3] *MPU9250_WE*, Wollewald, GitHub, 2024.
<https://github.com/rpicopter/ArduinoMotionSensorExample.git>
- [4] *AVR® Instruction Set Manual*, Microchip Technology Inc, 2021.
- [5] “Arduino mega2560 re v3.” Snapeda. <https://www.snapeda.com/parts/ATmega2560>
- [6] *Elegoo mega 2560 R3*, ELEGOO, 2025.
- [7] *MPU9250-9-Axis Sensor Module – Part 1* <https://wolles-elektronikkiste.de/en/mpu9250-9-axis-sensor-module-part-1>
- [8] *MPU-9250 Register Map and Descriptions*, InvenSense, 2013
- [9] *MPU-9250 Product Specification*, InvenSense, 2016
- [10] *Simple Snake Game in Python 3 for Beginners* TokyoEdTech.